RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop

Technical Documentation

Internship Project | Batch: adv-genai-feb-2026

Submitted by: Tanishka Prasad Jadhav
Intern ID: IN226010302

Table of Contents

Sr. No.	Section / Subsection	Page No.
1	Introduction	1
1.1	What is RAG?	1
1.2	Why RAG Is Needed	1
1.3	Use Case Overview	1
2	System Architecture Explanation	1
2.1	High-Level Architecture	1
2.2	Component Interactions	1
3	Design Decisions	1
3.1	Chunk Size Choice — 500 Characters, 50 Overlap	1
3.2	Embedding Strategy	1
3.3	Retrieval Approach — Top-K = 3	1
3.4	Prompt Design Logic	1
4	Workflow Explanation	1
4.1	LangGraph Usage	1
4.2	Node Responsibilities	1
4.3	State Transitions	1
5	Conditional Logic	1
5.1	Intent Detection	1
5.2	Routing Decisions	1
6	HITL Implementation	1
6.1	Role of Human Intervention	1
6.2	Current vs Future HITL Design	1
6.3	Benefits and Limitations	1
6.3.1	Benefits	1
6.3.2	Limitations	1
7	Challenges & Trade-offs	1
7.1	Retrieval Accuracy vs Speed	1
7.2	Chunk Size vs Context Quality	1
7.3	Cost vs Performance	1
8	Testing Strategy	1
8.1	Testing Approach	1
8.2	Sample Query Test Cases	1

8.3	Testing the HITL Threshold	1
9	Future Enhancements	1
9.1	Multi-Document Support	1
9.2	Feedback Loop	1
9.3	Memory Integration	1
9.4	Deployment	1
9.5	Production Architecture Upgrades	1

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that enhances Large Language Model (LLM) outputs by grounding them in external, domain-specific knowledge. Unlike pure LLMs that rely solely on training data, RAG systems retrieve relevant information from a knowledge base at query time and use it as context for generation.

User Query → Retrieve relevant documents → Feed as LLM context → Generate grounded answer

RAG consists of two fundamental phases:

- **Retrieval Phase:** A retriever converts the user query into an embedding and searches a vector database for semantically similar document chunks.
- **Generation Phase:** The retrieved chunks are injected into an LLM prompt as context. The LLM uses this context—rather than its own memory—to generate a precise answer.

The key advantage of RAG over fine-tuning is that the knowledge base can be updated without retraining the model. Simply re-ingest new documents into the vector store.

1.2 Why RAG Is Needed

Traditional LLMs have two major limitations that make them unsuitable for customer support applications:

- **Knowledge Cutoff:** LLMs are trained on data up to a certain date. They cannot answer questions about your specific product, policy, or documentation.
- **Hallucination Risk:** Without grounding, LLMs generate plausible-sounding but potentially incorrect answers — catastrophic for support contexts.

Comparison Dimension	Pure LLM	Fine-Tuned LLM	RAG System
Requires retraining?	No	Yes (expensive)	No
Domain-specific accuracy	Low	High	High
Updates knowledge?	Never	Periodic retraining	Re-ingest documents
Hallucination risk	High	Medium	Low (grounded)
Implementation complexity	Low	Very High	Medium
Cost	Low	Very High	Medium

1.3 Use Case Overview

This system is built for the Customer Support domain. The target use case is: a company uploads its product manuals, FAQ documents, or policy PDFs as a knowledge base. Users (customers or support agents) can then ask natural language questions and receive accurate, document-grounded answers.

- A customer asks: 'What is the return policy for electronics?' → System retrieves return policy section → LLM formulates clear answer
- A customer asks: 'How do I reset my device?' → System retrieves troubleshooting guide → LLM provides step-by-step answer
- A customer asks an off-topic question → No relevant chunks found → System escalates to human agent (HITL)

2. System Architecture

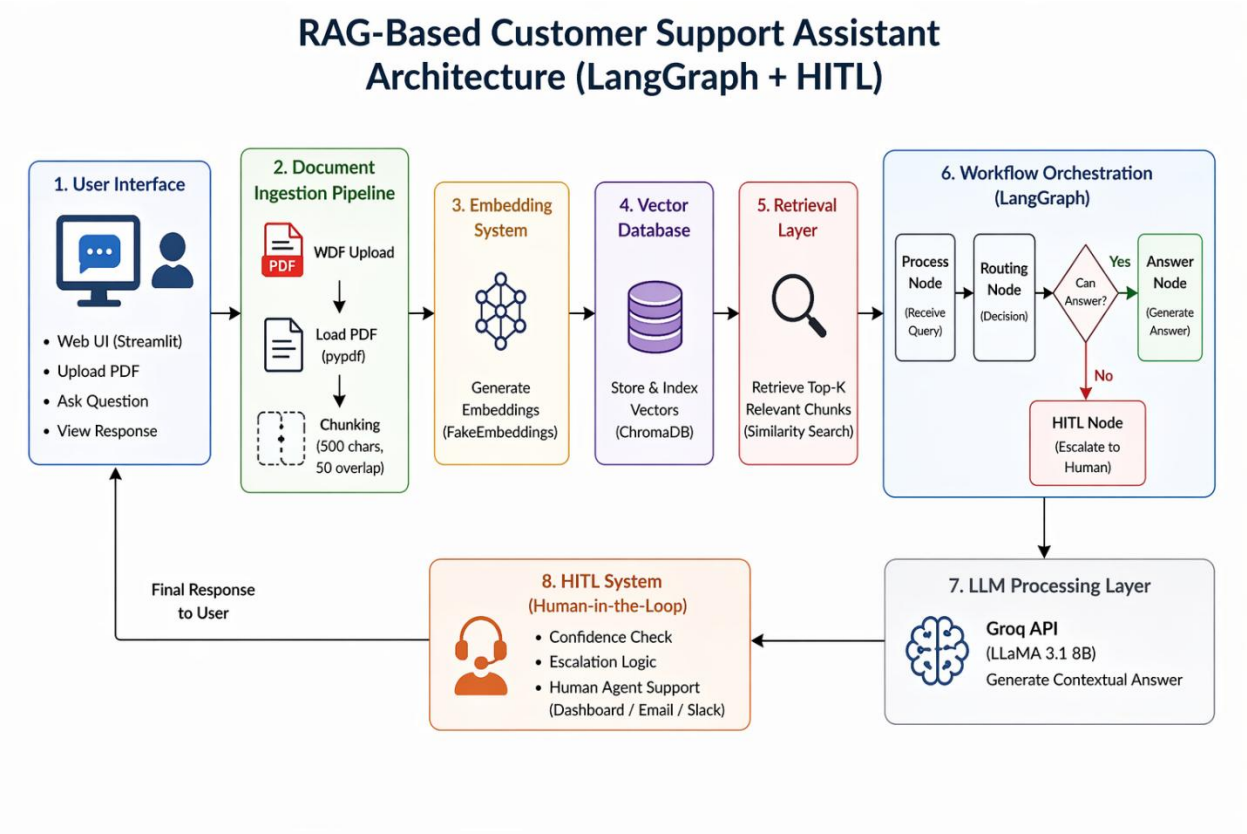

Fig 2.1 : System Architecture

2.1 High-Level Architecture

The system is organized into five logical layers, each with clear responsibilities:

Layer	Components and Responsibility
Presentation Layer	Streamlit web app — PDF upload, query input, response display, chat history
Ingestion Layer	pypdf (load_pdf), chunk_text — breaks PDF into 500-char chunks with 50-char overlap
Storage Layer	FakeEmbeddings + ChromaDB — vectorizes and stores chunks for fast retrieval
Orchestration Layer	LangGraph StateGraph — process → conditional routing → answer/hitl nodes
Intelligence Layer	Groq API (LLaMA 3.1 8B) — context-grounded answer generation

2.2 Component Interactions

The components interact in a strict sequential pipeline during document ingestion, and in a graph-based workflow during query processing:

During Ingestion: The Streamlit app receives the PDF, writes it to temp.pdf, calls load_pdf() to extract text, calls chunk_text() to split it, calls create_vector_store() to embed and store chunks, and finally calls get_retriever() to create a retriever object stored in session state.

During Query Processing: The app calls graph.invoke({'question': query}). LangGraph executes the process node, which calls retriever.invoke(question) to fetch top-3 relevant chunks. State['context'] is assembled and state['route'] is set. The conditional edge routes to either the answer node (which calls generate_answer()) or the hitl node. The response is returned from state['response'].

3. Design Decisions

3.1 Chunk Size Choice : 500 Characters, 50 Overlap

The chunking strategy uses 500-character fixed-size chunks with 50-character overlap. This decision was made based on the following reasoning:

Chunk Size	Pros	Cons	Decision
Small (100-200 chars)	Granular retrieval	Loses sentence context	Rejected
Medium (500 chars)	Balances context and precision	May split mid-sentence	Selected
Large (1000-2000 chars)	Full paragraphs preserved	Retrieval less precise	Future option

The 50-character overlap ensures that information at chunk boundaries is not lost. For example, a critical sentence that spans two chunks will still be present in both, ensuring it can be retrieved.

3.2 Embedding Strategy

The current implementation uses FakeEmbeddings (size=384) which generates random vectors. This was chosen for development and testing speed — no model download required. However, it means retrieval is random, not semantic.

Production Note:

For production, replace FakeEmbeddings with HuggingFaceEmbeddings('all-MiniLM-L6-v2').

This model generates 384-dimensional semantic embeddings and is free, fast, and widely used.

```
# Development (current):
from langchain_community.embeddings import FakeEmbeddings
embedding = FakeEmbeddings(size=384)

# Production (recommended):
from langchain_community.embeddings import HuggingFaceEmbeddings
embedding = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

3.3 Retrieval Approach Top-K=3

The retriever is configured to return the top 3 most similar chunks. This provides sufficient context for the LLM without overwhelming the prompt with noise. The cosine similarity search in ChromaDB ensures semantic relevance.

The context assembly uses a simple join: context = ' '.join([doc.page_content for doc in docs]). This produces a single context string of approximately 1500 characters (3 × 500), well within the LLaMA 3.1 context window of 128K tokens.

3.4 Prompt Design Logic

The prompt is engineered with specific rules to prevent hallucination and improve answer quality:

- **Role assignment:** 'You are a professional customer support assistant' - sets professional tone
- **Strict context rule:** 'Answer ONLY using the provided context' — prevents hallucination
- **Fallback rule:** Explicit instruction to say 'The document does not contain enough information' if answer not found
- **Format guidance**: 'Use bullet points if needed' — improves readability of structured answers
- **Low temperature (0.2):** Reduces randomness, increases factual consistency

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is a library by LangChain for building stateful, multi-step AI workflows as explicit directed graphs. Unlike standard LangChain chains (which are sequential), LangGraph supports branching, cycles, and conditional logic - making it ideal for agent-like systems.

In this project, LangGraph is used to build a 3-node workflow that explicitly separates retrieval logic, answer generation, and human escalation into distinct nodes with clear routing between them.

4.2 Node Responsibilities

Node	Responsibility	Input Keys	Output Keys	Next Node
process	Retrieve chunks, assemble context, determine routing	question	context, route	answer OR hitl
answer	Call Groq LLM with context and question	context, question	response	FINISH
hitl	Return escalation message to user	question	response	FINISH

4.3 State Transitions

The state object is a Python dict. Each node reads from state and writes to state:

```
# Initial state (user submits query)
state = {"question": "What is the return policy?"}
# After process node
state = {
    "question": "What is the return policy?",
    "context": "Section 3: Returns...within 30 days...original receipt...",
    "route": "ANSWER"
}
# After answer node (if route == ANSWER)
state = {
    "question": "What is the return policy?",
    "context": "Section 3: Returns...within 30 days...",
    "route": "ANSWER",
    "response": "Based on the document:\n• Returns are accepted within 30 days...\n• Original receipt required..."
}
# After hitl node (if route == HITL)
state = {
    "question": "What is quantum entanglement?",
    "context": "",
    "route": "HITL",
    "response": "Escalated to human support."
}
```

5. Conditional Logic

5.1 Intent Detection

The current system uses a simple heuristic for intent detection: context length. If the retrieved context is substantive (>= 50 characters), the system infers the question is within the scope of the document and proceeds to LLM generation. If the context is minimal or empty (< 50 characters), the system infers the question is out of scope or the document lacks relevant information.

Limitation:
This is not true intent classification — it is a proxy based on retrieval quality. A production system would use a dedicated intent classifier or confidence scoring on the retrieved embeddings.

5.2 Routing Decisions

The routing decision is made in the process node using a single conditional expression:

```
state["route"] = "HITL" if len(context.strip()) < 50 else "ANSWER"
```

The conditional edge then maps this route string to a node:

```
builder.add_conditional_edges(
    "process",
    lambda x: x["route"],
    {"ANSWER": "answer", "HITL": "hitl"}
)
```

This separation of routing logic (in process node) from routing execution (in conditional_edges) is a key LangGraph design pattern. It makes the routing decision visible and testable independently of the graph execution.

6. HITL Implementation

6.1 Role of Human Intervention

HITL (Human-in-the-Loop) is a design pattern in AI systems that keeps a human in the decision-making process for cases where the AI cannot confidently respond. In this customer support system, HITL serves as the safety net for:

- Queries outside the scope of the uploaded document
- Complex multi-part queries requiring judgment beyond simple retrieval
- Queries in different languages than the document
- Sensitive queries (complaints, legal issues) requiring human empathy

6.2 Current vs Future HITL Design

Aspect	Current (v1.0)	Future (v2.0)
Trigger	context length < 50 chars	Confidence score + intent classifier + keyword rules
Action on trigger	Set response = 'Escalated to human support.'	Create ticket, send email, notify via Slack
User feedback	Static message in chat UI	Ticket ID, ETA, agent name shown to user
Human response integration	Not implemented	Ticket system response fed back into chat
Logging	Not implemented	Unanswered queries logged to SQLite/MongoDB
Analytics	Not implemented	Dashboard of escalation rate, common HITL triggers

6.3 Benefits and Limitations

Benefits

- Prevents silent failures — no query is dropped without user notification
- Builds trust — users know a human will handle edge cases
- Enables continuous improvement — HITL logs identify gaps in the knowledge base
- Keeps AI scope bounded — prevents hallucination on out-of-scope queries

Limitations

- Current threshold (50 chars) is arbitrary — may trigger HITL for valid short contexts
- No actual human agent integration — escalation is purely informational in v1.0
- No confidence scoring — routing is binary, not probabilistic
- No feedback loop — human responses do not improve the system over time (yet)

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs Speed

Approach	Retrieval Accuracy	Speed	Chosen?
FakeEmbeddings	None (random)	Instant	Yes (dev only)
all-MiniLM-L6-v2	High	Fast (~50ms)	Recommended for prod
text-embedding-3-small (OpenAI)	Very High	Medium (API call)	Future option
FAISS + sentence-transformers	High	Very Fast	Alternative to ChromaDB

For production, sentence-transformers provides the best accuracy-to-speed ratio at zero cost.

7.2 Chunk Size vs Context Quality

Smaller chunks give more precise retrieval (less noise per chunk) but may miss important context that spans multiple sentences. Larger chunks preserve narrative context but make it harder to retrieve precisely relevant content for a specific question.

The 500-character chunk with 50-character overlap is a balanced default. For production, semantic chunking (splitting on sentence/paragraph boundaries) would significantly improve both precision and context quality.

7.3 Cost vs Performance

Component	Current Choice	Cost	Performance	Trade-off
LLM	Groq LLaMA 3.1 8B	Free tier	Good	Speed over accuracy
Embeddings	FakeEmbeddings	Free	None (random)	Dev speed over quality
Vector DB	ChromaDB in-memory	Free	Good for < 10K docs	Simplicity over scale
UI	Streamlit	Free	Good for prototyping	Speed over customization

8. Testing Strategy

8.1 Testing Approach

The system should be tested at three levels:

- **Unit Testing:** Each module function tested in isolation (load_pdf, chunk_text, create_vector_store, generate_answer)
- **Integration Testing:** Full pipeline test — upload PDF → query → verify response comes from document content
- **HITL Testing:** Verify escalation triggers correctly for out-of-scope queries

8.2 Sample Query Test Cases

Test ID	Query	Expected Route	Expected Response Type	Pass Condition
TC-01	What is the return policy?	ANSWER	Document-grounded answer	Response mentions return terms from document
TC-02	How do I contact support?	ANSWER	Contact information from doc	Response includes contact details
TC-03	What is quantum physics?	HITL	Escalation message	Response = 'Escalated to human support.'
TC-04	What are the payment options?	ANSWER	Payment info from doc	Accurate payment terms returned
TC-05	[Upload empty PDF]	HITL	Escalation for all queries	All queries escalated correctly
TC-06	[Upload image-only PDF]	HITL	Escalation for all queries	load_pdf returns empty string, HITL triggered
TC-07	Tell me a joke	HITL	Escalation message	Off-topic query correctly escalated

8.3 Testing the HITL Threshold

```
# Test the routing logic in isolation:
def test_routing():
    # Simulate process node behavior

    # Case 1: Good context → ANSWER
    context_good = "Returns are accepted within 30 days of purchase. Items must be in original condition."
    route = "HITL" if len(context_good.strip()) < 50 else "ANSWER"
    assert route == "ANSWER", "TC-ROUTE-01 FAILED"

    # Case 2: Empty context → HITL
    context_empty = ""
    route = "HITL" if len(context_empty.strip()) < 50 else "ANSWER"
    assert route == "HITL", "TC-ROUTE-02 FAILED"

    # Case 3: Near-threshold context (49 chars) → HITL
    context_short = "Returns: 30 days. Contact: info@co.com."   # 39 chars
    route = "HITL" if len(context_short.strip()) < 50 else "ANSWER"
    assert route == "HITL", "TC-ROUTE-03 FAILED"

    print("All routing tests passed!")
```

9. Future Enhancements

9.1 Multi-Document Support

The current system supports only a single PDF per session. Production systems need to handle multiple documents simultaneously. This can be achieved using ChromaDB collections — one collection per document or topic category, with a routing layer that determines which collection to query based on the user's query domain.

```
# Multi-document ChromaDB setup
vectordb = Chroma(
    collection_name="company_documents",
    embedding_function=HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2"),
    persist_directory="./chroma_db"
)

# Ingest multiple documents
for pdf_path in pdf_paths:
    text = load_pdf(pdf_path)
    chunks = chunk_text(text)
    vectordb.add_texts(texts=chunks, metadatas=[{"source": pdf_path}] * len(chunks))
```

9.2 Feedback Loop

A feedback mechanism allows users to rate responses (thumbs up/down). Negative-rated responses, combined with their queries and contexts, form a training signal to improve the chunking strategy, retrieval parameters, and prompt design over time. This closes the loop between deployment and continuous improvement.

9.3 Memory Integration

The current system has no conversation memory — each query is processed independently. LangChain's ConversationBufferMemory or LangGraph's persistent state can be used to maintain multi-turn context. This enables follow-up questions like 'Can you elaborate on point 3?' to work correctly.

9.4 Deployment

Deployment Option	Platform	Effort	Scale	Best For
Local	localhost:8501	Zero	Single user	Development
Cloud (Streamlit Cloud)	streamlit.io	Low	Small teams	Internal tools
Docker + AWS/GCP	ECS / Cloud Run	Medium	Production	Customer-facing
FastAPI backend + React frontend	Any cloud	High	Enterprise	Full product

9.5 Production Architecture Upgrades

- Replace FakeEmbeddings with all-MiniLM-L6-v2 for real semantic retrieval
- Enable ChromaDB persistence (persist_directory) to avoid re-ingestion on restart
- Add real HITL ticket integration (Zendesk, Freshdesk, or email)
- Implement conversation memory for multi-turn Q&A sessions
- Add a feedback widget (thumbs up/down) per response
- Add authentication (Streamlit-authenticator) for multi-user scenarios
- Add analytics dashboard showing query volume, HITL rate, response quality